

講義 2：變數與 TYPE / LIKE / CONSTANTS

對應練習：[ex02](#) | 答案程式：[ZR_TR02_TYPE_LIKE](#)

本講重點

- **DATA** 宣告變數：TYPE、LENGTH、DECIMALS、VALUE
- ABAP 內建資料型別總表與各自的適用場景
- **TYPE** (參考型別) 與 **LIKE** (參考另一個變數) 的差別
- **CONSTANTS** 常數
- 賦值 =、**CLEAR**，以及自動型別轉換的行為

1. 宣告變數：DATA

DATA 變數名 TYPE 型別 [LENGTH 長度] [DECIMALS 小數位] [VALUE 初始值].

```
DATA gv_name TYPE c LENGTH 10 VALUE 'ABAP'. " 固定長度文字
DATA gv_count TYPE i VALUE 100. " 整數
DATA gv_price TYPE p LENGTH 8 DECIMALS 2. " 帶小數的金額
DATA gv_memo TYPE string. " 變動長度文字

* 鏈式寫法 (實務標準寫法) :
DATA: gv_qty TYPE i,
      gv_today TYPE d.
```

命名慣例 (本課程與團隊風格一致)：全域變數 **gv_** (value)、**gs_** (structure)、**gt_** (table)；區域變數換成 **lv_** / **ls_** / **lt_**。前綴讓人一眼看出「這是什麼形狀的資料」。

2. 內建資料型別總表

型別	名稱	預設長度	初始值	用途與注意
c	固定長度字元	1	空白	不給 LENGTH 就只有 1 個字元，超長會被截斷
n	數字字元	1	'0...0'	存「長得像數字的編號」(如 0001)，不能拿來運算
string	變動長度字串	-	空字串	長度自動伸縮，一般文字建議用它
i	整數	4 bytes	0	計數、筆數
p	壓縮十進位	8 bytes	0	金額、數量的標準型別，搭配 DECIMALS 精確小數

型別	名稱	預設長度	初始值	用途與注意
f	浮點數	8 bytes	0.0	科學計算用，有精度誤差， 金額禁用
d	日期	8 字元	'00000000'	格式 YYYYMMDD，可直接加減天數
t	時間	6 字元	'000000'	格式 HHMMSS
x / xstring	十六進位	-	-	二進位資料（檔案內容），先認識即可

三個最容易踩的點：

- **TYPE c** 忘了 **LENGTH**：**DATA gv_x TYPE c.** 只能存 1 個字，塞 'ABC' 只剩 'A'。
- 金額用 **f**：浮點有二進位精度誤差（0.1 存不準），金額一律 **p ... DECIMALS n**。
- **n** 型別參與運算：它本質是字元，先轉成 **i/p** 再算。

日期運算範例（**d** 可以直接加減）：

```
DATA: gv_today TYPE d,
      gv_due    TYPE d.
gv_today = sy-datum.      " 系統欄位：今天日期
gv_due    = gv_today + 30. " 30 天後
WRITE: / '今天：', gv_today, '30 天後：', gv_due.
```

3. TYPE 與 LIKE 的差別

寫法	參考對象	讀法
DATA a TYPE i.	型別（內建型別、TYPES 定義、DDIC 型別）	「a 的型別是 i」
DATA b LIKE a.	另一個資料物件（變數、系統欄位、表格）	「b 跟 a 長一樣」

```
DATA gv_date1 TYPE d.      " TYPE：參考型別
DATA gv_date2 LIKE sy-datum. " LIKE：參考既有變數（系統欄位）
DATA gv_date3 LIKE gv_date1. " LIKE：跟 gv_date1 同型別
```

使用時機：

- 想表達「跟某個既有欄位 / 變數保持一致」用 **LIKE**——來源改了，跟著的變數自動一致。
- 其他情況用 **TYPE**。之後接觸 SAP 資料表後，**TYPE scarr-carrid**（參考資料字典欄位型別）會是最常見寫法（講義 6）。
- 進階常用：**DATA gs LIKE LINE OF gt_tab.**（宣告跟某內表一列同型別的 work area，講義 10 會用到）。

4. CONSTANTS 常數

宣告後不可改值的資料物件，**必須**給 **VALUE**：

```

CONSTANTS gc_max_score TYPE i VALUE 100.
CONSTANTS gc_pass_line TYPE i VALUE 60.

IF gv_score >= gc_pass_line.
    WRITE / '及格'.
ENDIF.

```

用途：把散落在程式裡的「魔術數字」「魔術字串」集中命名。改規則時只改一處，而且 `gc_pass_line` 比裸寫 `60` 可讀得多。對常數賦值會直接編譯錯誤，這是保護不是限制。

5. 賦值、CLEAR 與型別轉換

```

gv_count = 42.           " 賦值就是等號
gv_name  = 'HELLO'.
CLEAR gv_count.          " 恢復成該型別的初始值 ( i 是 0, c 是空白 )

```

不同型別互相賦值時，ABAP 會**自動轉換**（依固定的轉換規則），這很方便也很危險：

```

DATA: gv_char TYPE c LENGTH 3 VALUE '123',
      gv_int  TYPE i.
gv_int = gv_char.       " 自動 '123' → 123, 成功
gv_char = 'ABC'.
* gv_int = gv_char.     " 執行期當掉 ( CONV_NO_NUMBER ) — 內容不是數字

```

鐵律：轉換錯誤是**執行期**才爆，語法檢查抓不到。來源資料不可信（例如使用者輸入）時，賦值前要自己驗證。

6. 系統欄位初探 (structure SY)

系統隨時維護一組欄位，用 `sy-欄位名` 取用，本課程常用的：

欄位	內容
<code>sy-datum</code>	今天日期 (伺服器)
<code>sy-uzeit</code>	現在時間
<code>sy-uname</code>	登入帳號
<code>sy-subrc</code>	上一個操作的回傳碼 (0 = 成功)，講義 4 起天天用
<code>sy-tabix</code>	內表迴圈目前筆數 (講義 4)
<code>sy-index</code>	DO 迴圈目前圈數

7. 常見錯誤與陷阱

症狀	原因
文字被截成 1 個字	<code>TYPE c</code> 沒給 <code>LENGTH</code>
金額加總出現 0.0000001 誤差	用了 <code>f</code> ，該用 <code>p DECIMALS</code>
對 <code>CONSTANTS</code> 賦值編譯錯誤	常數本來就不可改——改宣告的 <code>VALUE</code>
執行期 <code>CONVT_NO_NUMBER</code> 當掉	字元轉數值但內容不是數字
<code>LIKE</code> 後面接型別名報錯	<code>LIKE</code> 只能接資料物件，接型別要用 <code>TYPE</code>

8. 課堂練習

完成 [ex02](#)：宣告各型別變數、用 `LIKE` 參考、定義常數並觀察轉換行為。